

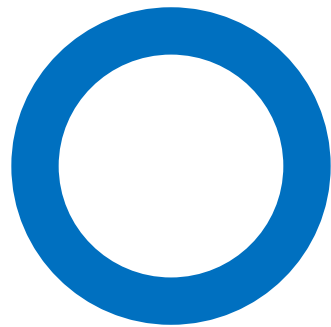
# Polymorphism

Lập trình hướng đối tượng

Hồ Tuấn Thanh

[htthanh@fit.hcmus.edu.vn](mailto:htthanh@fit.hcmus.edu.vn)

# Con trỏ



# Thần chú

- MIỀN GIÁ TRỊ CỦA MỘT BIẾN CON TRỞ LÀ ĐỊA CHỈ Ô NHỚ

# Toán tử

- &: Lấy địa chỉ của một biến
- \*: Lấy giá trị tại ô nhớ mà con trỏ trỏ đến

# Lưu ý

- ❑ new: cấp vùng nhớ mới cho con trỏ quản lí
- ❑ delete: xóa vùng nhớ đã cấp cho con trỏ
- ❑ Có new thì phải có delete
  - Cấp phát thì phải thu hồi để cho chương trình khác sử dụng (memory leaking)
- ❑ Vùng nhớ mồ côi: vùng nhớ không còn con trỏ nào tham chiếu đến nữa
- ❑ Rò rỉ bộ nhớ (memory leaking): không delete sau khi đã new và sử dụng xong

# Con trỏ

```
GiaoVien gv("Ho Tuan Thanh", 20000000, 1);
GiaoVien *p;
// ĐÚNG: p và gv cùng là GiaoVien
p=&gv;

GVCN gvcn("Le Ha Minh", 3000000, 2, "12CK2");
// ĐÚNG: con trỏ lớp cha ĐƯỢC QUYỀN tham chiếu lớp con
p=&gvcn;

GVCN *p2;
// ĐÚNG: p2 và gvcn cùng là GVCN
p2=&gvcn;
// SAI: con trỏ lớp con KHÔNG ĐƯỢC QUYỀN tham chiếu lớp cha
p2=&gv;
```

# Con trỏ

```
GiaoVien *p;
```

Ham ao

Lien ket dong

Lam sao goi duoc  
ham Xuat lop GVCN???

```
GVCN gvcn("Le Ha Minh", 3000000, 2, "12C12");
```

```
// ĐÚNG: con trỏ lớp cha ĐƯỢC QUYỀN tham chiếu lớp con  
p=&gvcn;
```

Lien ket tinh

```
// Gọi hàm xuất của lớp GiaoVien
```

```
P->Xuat();
```

Con tro => goi thuoc tinh/ham dung dau ->, ko dung dau .

# Con trỏ

```
void f1(GiaoVien gv)
{
    gv.Xuat();
}
```

Goi ham Xuat cua  
lop GiaoVien

```
void main()
{
    GVCN gvcn("Le Ha Minh", 3000000, 2, "12CK2");
    // OK: Tham số là đối tượng lớp cha,
    // được quyền truyền đối tượng lớp con vào
    f1(gvcn);
}
```

# Con trỏ

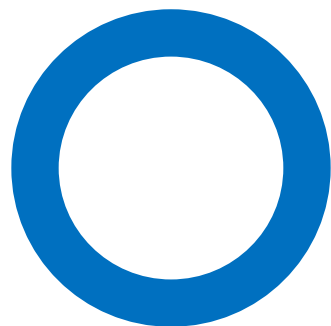
```
void f2(GiaoVien* p)
{
    p->Xuat();
}

void main()
{
    GVCN gvcn("Le Ha Minh", 3000000, 2, "12CK2");
    // OK: Tham số là đối tượng lớp cha,
    // được quyền truyền đối tượng lớp con vào
    f2(&gvcn);
}
```

Diagram illustrating the relationship between the code and the call:

- `p->Xuat();` in `f2` is linked to **Ham ao**.
- `f2(&gvcn);` in `main` is linked to **Lien ket tinh**.
- Lien ket tinh** is linked to **Goi ham Xuat lop GiaoVien**.
- Goi ham Xuat lop GiaoVien** is linked to **Lam sao goi duoc ham Xuat lop GVCN**.
- Lam sao goi duoc ham Xuat lop GVCN** is linked to **Lien ket dong**.
- Lien ket dong** is linked back to **Ham ao**.

# Hàm ảo



# Khái niệm hàm ảo

## ❑ Là một hàm trong lớp

- Hàm friend không thể làm hàm ảo

## ❑ Mang tính chất ảo

- Gọi đúng hàm mà con trỏ đang trỏ đến
- Đây được gọi là cơ chế liên kết động

Thường được cài đặt bởi vtable (virtual function table)

Bài tập: tìm hiểu vtable, viết report 2 trang theo định dạng Markdown

- Tính chất thứ 3 của OOP: **tính đa hình**

## ❑ Khai báo hàm ảo trong C++:

- Thêm từ khóa **virtual** ở trước khai báo hàm

# Ví dụ - không sử dụng hàm ảo

```
class Animal
{
public:
    void talk()
    {
        cout<<"Biet keu gi day???" ;
    }
};
```

```
class Cat: public Animal
{
public:
    void talk()
    {
        cout<<"Meo meo!!!";
    }
};
```

```
class Dog: public Animal
{
public:
    void talk()
    {
        cout<<"Gau gau!!!";
    }
};
```

# Ví dụ - không sử dụng hàm ảo

```
void main()
{
    Animal a;
    Cat c;
    Dog d;
    Animal *p;

    p=&a;
    p->talk(); // Biet keu gi day???

    p=&c;
    p->talk(); // Biet keu gi day???

    p=&d;
    p->talk(); // Biet keu gi day???
}
```

# Ví dụ - sử dụng hàm ảo

```
class Animal
{
public:
    virtual void talk()
    {
        cout<<"Biet keu gi day???" ;
    }
};
```

Chi viet tu virtual o lop cha  
Khong viet o lop con

An tien la o cho nay!!!

```
class Cat: public Animal
{
public:
    void talk()
    {
        cout<<"Meo meo!!!";
    }
};
```

```
class Dog: public Animal
{
public:
    void talk()
    {
        cout<<"Gau gau!!!";
    }
};
```

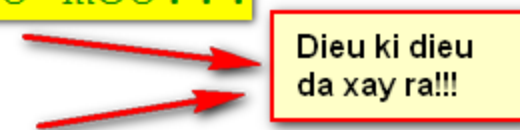
# Ví dụ - sử dụng hàm ảo

```
void main()
{
    Animal a;
    Cat c;
    Dog d;
    Animal *p;

    p=&a;
    p->talk(); // Biet keu gi day???

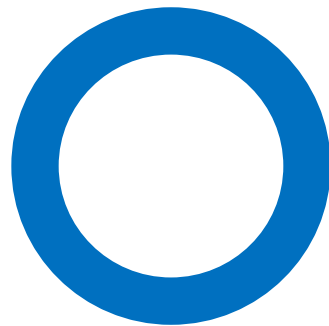
    p=&c;
    p->talk(); // Meo meo!!!

    p=&d;
    p->talk(); // Gau gau!!!
}
```



Dieu ki dieu  
da xay ra!!!

# Tính đa hình



# Đa hình là gì?

- ❑ The dictionary definition of polymorphism refers to a principle in biology in which an organism or species can have many different forms or stages.
- ❑ A *parent* class reference is used to *refer* to a *child* class object → A call to a member function will cause a different function to be executed *depending* on the *type of object* that invokes the function.

# Làm sao để đa hình?

- ☐ Khai báo kế thừa
- ☐ Khai báo hàm ảo ở lớp cha
- ☐ Sử dụng con trỏ

# Đa hình để làm gì?

- ❑ Chương trình viết ngắn hơn, dễ mở rộng hơn

# Lợi ích 1 – Viết ngắn hơn

```
void giveATalk(Animal obj, int type)
{
    if(type==1)
    {
        Cat c=(Cat)obj;
        c.talk();
    }
    else if(type==2)
    {
        Dog d=(Dog)obj;
        d.talk();
    }
}
```

KHÔNG SỬ DỤNG tính đa hình

```
void giveATalk(Animal *p)
{
    p->talk();
}
```

CÓ SỬ DỤNG tính đa hình

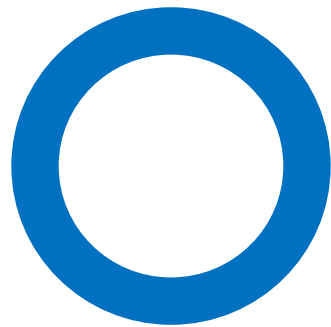
# Lợi ích 2 – Dễ mở rộng

```
void giveATalk(Animal obj, int type)
{
    if(type==1)
    {
        Cat c=(Cat) obj;
        c.talk();
    }
    else if(type==2)
    {
        Dog d=(Dog) obj;
        d.talk();
    }
    else if(type==3)
    {
        Mouse m=(Mouse) obj;
        m.talk();
    }
}
```

```
void giveATalk(Animal *p)
{
    p->talk();
}
```

CÓ SỬ DỤNG tính đa hình

# Hàm thuần ảo



# Khái niệm hàm thuần ảo

- ❑ Có những hàm ảo mà không biết cài đặt như thế nào ở lớp cha
- ➔ Biến chúng thành hàm thuần ảo
- ❑ Khai báo hàm thuần ảo trong C++:
  - **virtual** <Chữ ký hàm> = 0;
- ❑ Tiếng Anh: **pure virtual**

# Ví dụ

```
class Animal
{
public:
    virtual void talk();
};

void Animal::talk()
{
    cout<<"Biet keu gi day???";
```

KHONG SU DUNG ham thuan ao

# Ví dụ

```
class Animal
{
public:
    virtual void talk()=0;
};
```

SU DUNG ham thuan ao

```
void Animal::talk()
{
    cout<<"Biet keu gi day???";
```

# Lớp trừu tượng

- ❑ Là lớp chứa **MỘT** hàm thuần ảo
- ❑ Không thể tạo đối tượng từ lớp trừu tượng
- ❑ Chỉ dùng để kế thừa
- ❑ Lớp con kế thừa lớp trừu tượng:
  - Cần cài đặt **TẤT CẢ** các hàm thuần ảo
  - Nếu không, lớp con sẽ trở thành lớp trừu tượng
- ❑ Tiếng Anh: **abstract class**

```
class Animal
{
public:
    virtual void talk()=0;
};

class Duck: public Animal
{
};
```

```
void main()
{
    // SAI: Lớp trừu tượng
    Animal a;

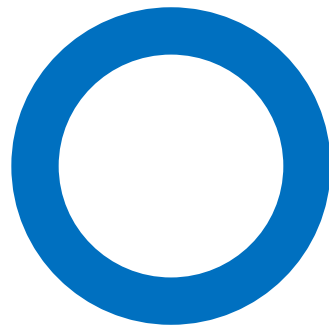
    // DUNG
    Animal *p;

    // DUNG
    Cat c;
    // DUNG
    p=&c;

    // SAI: Lớp trừu tượng
    p=new Animal;
    // DUNG
    p=new Cat;

    // SAI: Không cài đặt lại
    // hàm thuan ao lop cha
    Duck d;
}
```

# Hàm hủy ảo



# Ví dụ tuần trước

```
~Base() {  
    cout<<"Base destructor" << endl;  
}
```

```
Base*pb;  
pb=new DerivedB(4,5);  
delete pb;
```

HUY lóp Base

# Ví dụ tuần trước

- ❑ Làm sao để gọi hàm hủy lớp Derived B

```
virtual ~Base() {  
    cout<<"Base destructor" << endl;  
}
```

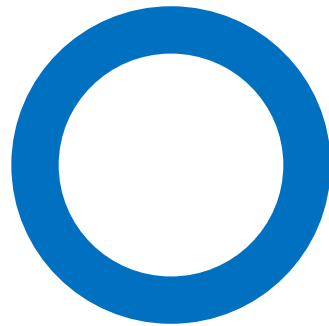
```
Base*pb;  
pb=new DerivedB(4,5);  
delete pb
```

Goi ham huy lop DerivedB  
Goi ham huy lop Base

# Hàm hủy ảo

- ❑ Hàm hủy của lớp phải luôn là hàm ảo
- ➔ Chuyển lời gọi hàm đến hàm hủy của lớp kế thừa

# Interface



# Interface

- ❑ As you've already learned, objects define their *interaction* with the outside world through the *methods* that they expose. Methods form the object's *interface* with the outside world.
- ❑ In OOP, the term interface is often used to define an abstract type that *contains no data or code, but defines behaviors* as method signatures.
  - A class having code and data for all the methods corresponding to that interface is said to *implement* that interface.

# Interface

- ❑ Interface là một class:
  - KHÔNG chứa thuộc tính
  - CHỈ CHỨA các hàm thuần ảo
- ❑ C++: không có từ khóa interface
- ❑ C#, Java, ...: có từ khóa interface

# Interface

- ☐ You are in the middle of a large, empty room, when a zombie suddenly attacks you.
- ☐ You have no weapon.
- ☐ Luckily, a fellow living human is standing in the doorway of the room.
- ☐ "Quick!" you shout at him. "Throw me something I can hit the zombie with!"

# Interface

- ❑ You didn't specify (nor do you care) exactly **what** your friend will choose to toss;...But it doesn't matter, as long as:
  - It's something that **can** be tossed (He can't toss you the sofa)
  - It's something that you can grab hold of (Let's hope he didn't toss a shuriken)
  - It's something you can use to bash the zombie's brains out (That rules out pillows and such)
- ❑ It doesn't matter whether you get a baseball bat or a hammer -  
as long as it implements your three conditions, you're good.
- ❑ **To sum it up:** When you write an interface, you're basically saying: "I need something that..."

# Interface

- ❑ Interface là một bảng mô tả tất cả các hành động mà một đối tượng có thể làm.
  - Ta không quan tâm nó làm thế nào, chỉ biết đây là những thứ mà đối tượng có thể làm.
- ❑ Trong OOP, interface là một bảng mô tả tất cả các hàm mà đối tượng phải có để nó được công nhận là X. Interface giống như một contract (hợp đồng).
- ❑ Tất cả những gì “ACT LIKE” một cái đèn đều phải thỏa interface với 2 hàm `turnOn()` và `turnOff()`.

# Interface

- ❑ Nội dung file header trong C/C++ chính là hình ảnh của interface.
  - File header chứa các khai báo hàm.
  - Chúng ta ko quan tâm hàm đó xử lí bên trong như thế nào.
  - Chúng ta chỉ quan tâm: hàm đó dùng để làm gì, cần tham số đầu vào là gì, làm xong trả ra kết quả gì.

# Interface

```
int TimLonNhat(int a, int b);  
int TimNhoNhat(int a, int b);  
  
bool KiemTraNguyenTo(int k);  
bool KiemTraChinhPhuong(int k);  
void LietKeUocSo(int k);  
  
int UCLN(int a, int b);  
int BCNN(int a, int b);  
  
int GiaiPTBacNhat(float a, float b, float& x);  
int GiaiPTBacHai(float a, float b, float c, float& x1, float& x2);  
  
void Nhap(PhanSo&p);  
void Xuat(PhanSo p);  
PhanSo Cong(PhanSo p, PhanSo q);
```

# Interface

- ❑ Tất cả bài tập bạn làm trong môn này đều có dạng yêu cầu sau: Tôi cần 1 cái gì đó mà có thể...
- ❑ Tôi cần 1 cái gì đó mà có thể
  - Nhập vào phân số
  - Xuất phân số ra màn hình
  - Cộng 2 phân số
  - Trừ 2 phân số
  - So sánh 2 phân số
- ❑ Chúng ta có **interface IPhanSo**
- ❑ Chúng ta có class PhanSo **cài đặt** interface IPhanSo
- ❑ Chúng ta ko quan tâm phân số đó được cài đặt thế nào

# Interface

- ❑ Tôi cần 1 cái gì đó mà có thể
  - Nhập vào một điểm trong mặt phẳng
  - Xuất điểm ra màn hình
  - Tính khoảng cách 2 điểm đưa vào
  - Cho biết điểm thuộc góc phần tư thứ mấy
- ❑ Chúng ta có **interface IDiem**
- ❑ Chúng ta có class Diem **cài đặt** interface IDiem
- ❑ Chúng ta ko quan tâm điểm đó được cài đặt thế nào

# Interface

- ❑ Tôi cần 1 cái gì đó mà có thể
  - Nhập vào một ngày tháng năm
  - Xuất ngày ra màn hình
  - Cho biết ngày mai sẽ là ngày mấy
  - Cho biết ngày nào trước ngày nào sau
- ❑ Chúng ta có **interface INgay**
- ❑ Chúng ta có class Ngay **cài đặt** interface INgay
- ❑ Chúng ta ko quan tâm ngày đó được cài đặt thế nào
  - Bạn dùng 3 số int để lưu ngày tháng năm hay 1 số int lưu khoảng cách đến 1/1/1 → MẶC KẾ
  - Bạn tính ngày kế tiếp như thế nào → MẶC KẾ

# Interface

- ❑ Tôi cần 1 cái gì đó mà có thể
  - Nhập vào một danh sách sinh viên
  - Xuất danh sách sinh viên ra màn hình
  - Cho biết sinh viên nào có ĐTB cao nhất
  - Cho biết sinh viên nào có ĐTB dưới 5
- ❑ Chúng ta có **interface IMangSinhVien**
- ❑ Chúng ta có class **SinhVien** cài đặt interface **IMangSinhVien**
- ❑ Chúng ta ko quan tâm mảng đó được cài đặt thế nào
  - Sử dụng mảng tĩnh SinhVien a[100] → MẶC KẾ
  - Sử dụng mảng động SinhVien \*a → MẶC KẾ
  - Sử dụng vector vector<SinhVien> a → MẶC KẾ

# Interface

- ❑ Tôi cần 1 cái gì đó mà có thể
  - Chứa dữ liệu dạng ngăn xếp
  - Thêm dữ liệu vào ngăn xếp: push
  - Lấy dữ liệu ra khỏi ngăn xếp: pop
  - Cho biết phần tử đầu ngăn xếp: peek
  - Cho biết ngăn xếp có rỗng ko: empty
- ❑ Chúng ta có **interface IStack**
- ❑ Chúng ta có class Stack **cài đặt** interface IStack
- ❑ Chúng ta ko quan tâm ngăn xếp đó được cài đặt thế nào
  - Sử dụng mảng tĩnh int a[100] → MẶC KỆ
  - Sử dụng danh sách liên kết → MẶC KỆ

# Ví dụ (C++)

```
class IIntArray
{
public:
    virtual void Input()=0;
    virtual void Output()=0;
    virtual int FindMax()=0;
};
```

```
class StaticIntArray: public IIntArray
{
private:
    int a[100];
    int n;
public:
    void Input();
    void Output();
    int FindMax();
};
```

# Ví dụ (C++)

```
class VectorIntArray: public IIntArray
{
private:
    vector<int> a;
public:
    void Input();
    void Output();
    int FindMax();
};
```

```
void main()
{
    IIntArray *arr;
    arr=new VectorIntArray;
    arr->Input();
    arr->Output();
    cout<<arr->FindMax();
}
```

